



FPGA-Based Real-Time Driver Drowsiness Detection Using RISC-V and Hardware Acceleration

S K Hari Keshav

Department of Computer Science
and Engineering, IIT Madras,
Chennai - 600036
cs24s025@cse.iitm.ac.in

K Srirag

School of Interdisciplinary
Studies, IIT Madras,
Chennai - 600036
id25s030@smail.iitm.ac.in

Saptajit Banerjee

School of Interdisciplinary
Studies, IIT Madras,
Chennai - 600036
id25s001@smail.iitm.ac.in

ABSTRACT

Driver drowsiness is a major cause of road accidents and creates the need for efficient real-time monitoring systems. This work presents a driver drowsiness detection system designed for deployment on the Shakti RISC-V FPGA platform using lightweight deep learning models and hardware-aware optimization techniques. Different CNN architectures were trained and evaluated, and a hybrid optimization pipeline combining TFMOT and ONNX optimization was used to reduce model size and inference latency while maintaining good classification accuracy. Among the tested models, MobileNetV3 Small provided the best overall performance, achieving 96.88% accuracy with low latency and reduced model size. The system also includes a TVM-based deployment flow, a custom FPGA convolution accelerator, and an OV2640 camera-based UART image transmission pipeline for embedded FPGA deployment on the Shakti RISC-V platform.

1. INTRODUCTION

Driver drowsiness is one of the major causes of road accidents, leading to reduced reaction time, poor decision-making, and loss of vehicle control. Real-time driver monitoring systems are therefore becoming an important component of modern automotive safety technologies. Recent advances in deep learning have enabled accurate vision-based drowsiness detection using facial features such as eye closure, yawning, and head movement. However, deploying such models on embedded edge platforms remains challenging due to limited computational resources, memory constraints, and strict real-time requirements.

FPGAs provide an effective platform for embedded AI applications because of their parallel processing capability, low power consumption, and hardware configurability. At the same time, the open-source RISC-V architecture enables flexible processor customization and accelerator integration for domain-specific workloads. Combining FPGA

acceleration with RISC-V processors offers a promising solution for efficient real-time driver monitoring systems.

In this work, we propose an FPGA-based real-time driver drowsiness detection system built on the Shakti RISC-V platform. The system integrates lightweight deep learning models, TVM-based deployment, and custom FPGA hardware acceleration for convolution operations. A hybrid optimization pipeline combining TensorFlow Model Optimization Toolkit (TFMOT) and ONNX optimization is developed to reduce model size while maintaining inference accuracy and low latency.

2. LITERATURE REVIEW

Mousavikia et al. [1] describes the design and implementation of a driver drowsiness detection system using a modified RISC-V processor on a field-programmable gate array (FPGA). To detect drowsiness, a basic Convolutional Neural Network (CNN) is implemented on a RISC-V processor. The CNN is trained to classify four primary driver expressions, including distraction, natural, sleep, and yawn. The trained CNN achieves 81.07% accuracy on the validation data. To bypass the FPGA memory limitations, the C code for the trained CNN is optimized in multiple ways. Some optimizations include the usage of dynamic fixed-point data types and dynamic memory allocations. Also the processor is modified by adding three custom instructions, including custom store, conv2d(2x2) and multiply and accumulation (MAC) to enhance the computation rate. The resultant processor with the aforementioned custom instructions achieved the best result in terms of latency, with an improvement factor of 1.7 over the base processor and 1.25 over the processor with only custom store, multiply, and accumulation (MAC) in exchange for a slight increase in area.

Reddy et al. [2] proposes a deep learning-based approach for driver drowsiness detection, which has been optimized for deployment on edge devices. The authors implement Multi-Task Cascaded Convolutional Networks (MTCNN)

¹ Project Repositories: Model Optimization Pipeline

(<https://www.kaggle.com/code/rogaldorn7/drowsiness-detection-model-optimization>)

Hardware Deployment & Execution (<https://github.com/Harikheshav/Drowsiness-Detection>)



for efficient detection and extraction of facial features, which are then passed to the drowsiness detection model. To enable real-time inference on resource-constrained hardware, the study utilizes Distillation Learning, where knowledge from a large teacher network is transferred to a small student network. This model compression technique greatly lowers computational complexity and model size. It maintains competitive accuracy, enabling the final system to run well on embedded platforms such as the NVIDIA Jetson TK1.

Dosovitskiy et al. [3] introduce CARLA for autonomous driving research. It was developed from the ground up to support development, training, and validation of autonomous urban driving systems. They provide open-source code, protocols, and open digital assets (urban layouts, buildings, vehicles) that were created for this purpose and can be used freely. The simulation platform supports the specification of sensor suites and environmental conditions. CARLA is used to study the performance of three approaches to autonomous driving: a classic modular pipeline, an end-to-end model trained via imitation learning, and an end-to-end model trained via reinforcement learning. These approaches are evaluated in controlled scenarios of increasing difficulty, and the performance can be examined via metrics provided internally by CARLA.

3. GAPS IN CURRENT STUDY

A limitation of the study done by Mousavikia et al [1] is that the system relies solely on camera-based input to detect the driver's state. No additional modal sensors are considered to support or validate the detection results. Incorporating other sensing modalities could improve the reliability and robustness of the driver monitoring system.

There is a gap in this study by B. Reddy et al. [2], which is that additional compression techniques, such as pruning, weight clustering, and quantization, were not implemented to further reduce the student model size and improve inference speed.

Although CARLA, which was extensively used by Dosovitskiy et al. [3] in their studies, is an efficient simulator for testing autonomous driving systems, it does not provide a dedicated framework specifically designed for driver drowsiness detection experiments. The platform mainly focuses on evaluating autonomous driving algorithms rather than monitoring driver behavior or fatigue-related states. As a result, additional frameworks or modules are required to effectively simulate and test driver drowsiness detection systems within the CARLA environment.

4. INDIVIDUAL ROLES AND CONTRIBUTIONS

S K Hari Keshav (CS24S025):

- Create the system for using a camera and read image data, using an IMU Sensor and read sensor data on a RISC-V-based CPU on an FPGA.
- Port the optimized model and image detection code to the above CPU.
- Run evaluations and find the bottleneck computation to accelerate.
- Create and test an accelerator to speed up the above computation individually and with the CPU.

Saptajit Banerjee (ID25S001):

- Designed, trained, tested and validated the Computer Vision-based drowsiness detection model
- Developed and evaluated three optimization strategies (TFMOT, ONNX, and a novel hybrid pipeline) to balance compression, latency, and accuracy
- Benchmarked five architectures (RTDD, ResNet-18, MobileNetV3 Small/Large, MobileNetV2) across throughput, latency, accuracy, and model size metrics, selecting MobileNetV3 Small as the most optimal model
- Tested all original, quantized (TFLite), and ONNX-optimized models on personal laptop; deployed and validated the baseline and quantized (TFLite) model versions on a Raspberry Pi for real-world edge inference
- Deployed the drowsiness detection model in CARLA and designed the real-time closed-loop driver intervention system, including MediaPipe-based face detection, 1-minute prediction buffer, 70% drowsiness threshold trigger, CARLA Traffic Manager integration for autonomous mode switching, and manual override functionality

K Srirag (ID25S030):

- Developed the OV2640 camera-based image acquisition and UART transmission pipeline.
- Implemented packetized image streaming with headers, sequence numbering, and CRC-based error checking.



- Designed FSM-based packet parser for continuous byte-stream processing and sequential image frame reconstruction.
- Optimized UART communication for low-bandwidth embedded transmission.

5. MILESTONES

Vision Model Development

- Train Light-Weight Drowsiness Detection Models
- Validate Detection Accuracy

Embedded Baseline

- Deploy a complete pipeline on RISC-V
- Measure latency and execution cycles.

Hardware Acceleration

- Identify computational Bottleneck
- Implement accelerator in FPGA logic
- Integrate via memory-mapped interface

Comparative Evaluation

- Software only vs accelerated
- Test in CARLA simulation environment

6. METHODOLOGY

6.1. IMAGE ACQUISITION AND TRANSMISSION PIPELINE

As part of the FPGA-integrated architecture, an image acquisition and transmission pipeline was developed using an Arducam Mini camera module based on the OV2640 image sensor which supports onboard JPEG compression, significantly reducing the bandwidth requirements for image transmission. The image frames are acquired and transmitted using a structured packet-based communication pipeline. The compressed image is divided into fixed-size payload chunks and packetized with headers and footers for synchronization, sequence identifiers, payload length, CRC validation, to ensure reliable frame transfer and integrity verification.

This image stream is then transmitted over a UART-based serial communication interface for effective low-bandwidth communication with the Shakti RISC-V processor. On the receiver side, a FSM based packet parser is implemented to continuously process the incoming byte-stream, and reconstruct the image frames sequentially. This architecture is designed to support integration with the Shakti RISC-V-based FPGA platform.

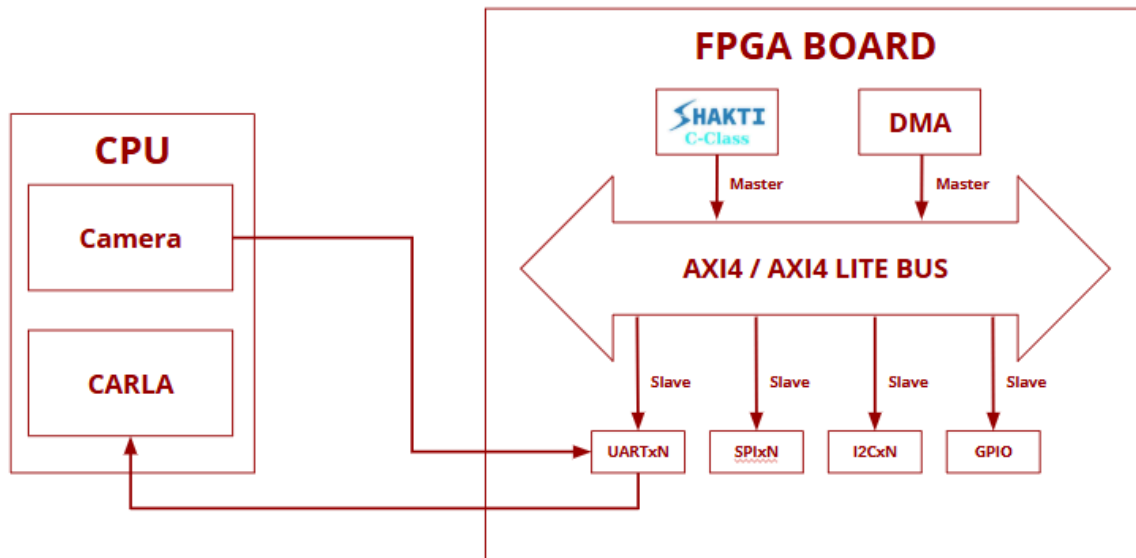


Figure 1: System architecture for the FPGA board

6.2. SYSTEM DESIGN OF CLOSED-LOOP SAFETY SYSTEM

The proposed architecture establishes a real-time, closed-loop safety intervention system utilizing the CARLA simulator. Upon system initialization, the ego-vehicle is deployed in manual driving mode, allowing the driver full control. In parallel, an asynchronous vision pipeline captures live video streams from a USB camera. The design of this front-end pipeline is inspired by the work of Reddy et al. [2], which utilized an MTCNN for face detection. To

improve upon this, we have replaced the computationally heavy MTCNN with MediaPipe's lightweight face_detector.tflite model.

This modification significantly reduces the computational overhead and latency of the pre-processing stage, allowing for a higher overall system throughput. The MediaPipe model detects faces and then extracts them from this live video stream. These faces are then fed to a drowsiness detection model, which then feeds the detections to a 1



Minute buffer, where these detections are collected and analyzed for a period of 1 minute.

The control system applies a strict heuristic threshold to this window: if 70% or more of the accumulated predictions classify the driver as 'drowsy', a state-change trigger is activated. This command interfaces directly with the CARLA Traffic Manager, revoking manual control and engaging the vehicle's autonomous safe-driving mode. To complete the closed-loop state machine, the system includes a manual override function. The operator can actively

disengage the autonomous mode to resume manual control, which automatically resets and re-initializes the drowsiness monitoring cycle.

Figure 2 illustrates the architectural flowchart of this closed-loop system. A practical demonstration of the system's execution, detailing the classification-to-actuation pipeline and the autonomous state transitions, is available in the [\[Supplementary Demo Video\]](#).

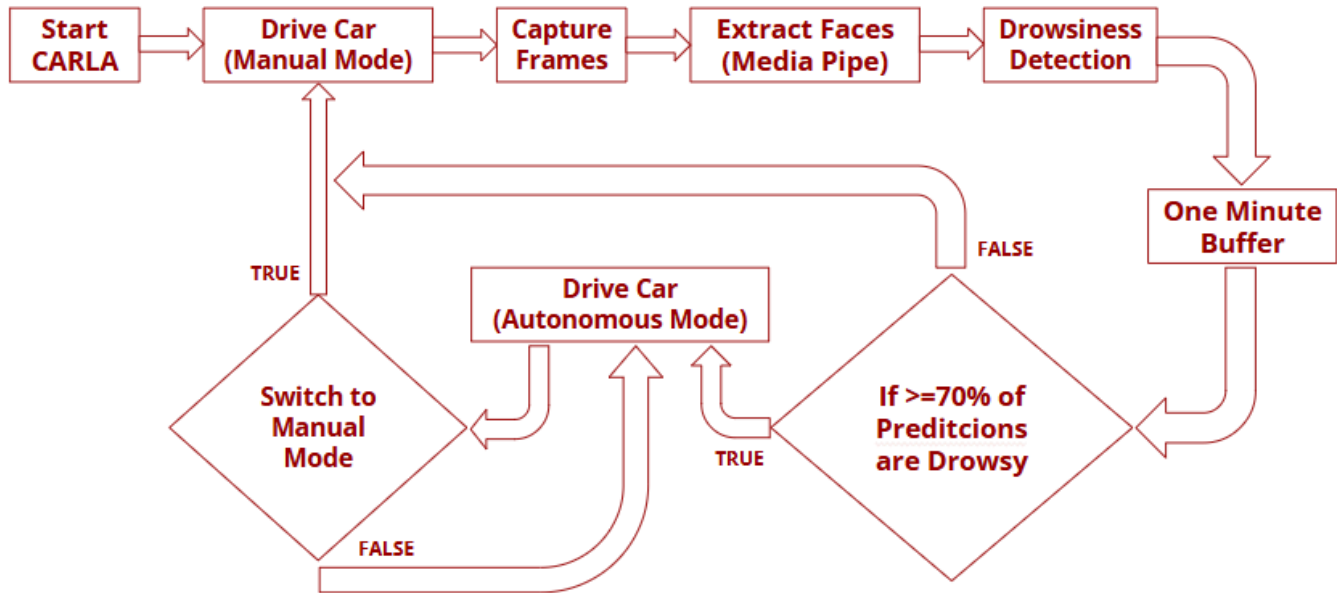


Figure 2: System architecture and execution flow for the closed-loop driver drowsiness intervention system

6.3. OPTIMIZATION PIPELINES

The baseline models were trained using the Driver Drowsiness Dataset (DDD) curated by Nasri [4], which provides diverse spatial representations of fatigue-related facial features. To establish an initial deep learning framework, the foundational training scripts were adapted from the baseline architecture provided by Sayed [5].

To optimize the drowsiness detection model for edge deployment, two distinct optimization pipeline strategies were evaluated: a size reduction pipeline utilizing the TensorFlow Model Optimization Toolkit (TFMOT), and a latency reduction ONNX optimization pipeline.

The Standard TFMOT Optimization Pipeline yielded 10.7x reduction in raw model size and 26.22x reduction in zipped model size. However, this came at a severe performance cost, with an observed 15.46x degradation in throughput and a corresponding 15.59x increase in latency, rendering the model unsuitable for real-time applications.

To address these performance bottlenecks, the ONNX optimization pipeline was subsequently implemented. This strategy proved effective, achieving an approximate 8x improvement in throughput and a 6.5x reduction in latency

while maintaining classification accuracy above 99% with a negligible impact on model size. A critical failure occurred, however, during the final compression step: applying full integer (INT8) static quantization to the ONNX-optimized model induced a catastrophic accuracy degradation, dropping from ~99% to ~55%. The complementary failures of these two pipelines—one succeeding in compression but failing in performance, the other succeeding in performance but failing in quantization—motivated the development of a hybrid approach. The architectural flow of these initial, distinct strategies are detailed in the flowcharts below.



Figure 3: The TFMOT Optimization Pipeline for Model Size Reduction.



Figure 4: The ONNX Optimization Pipeline for Latency Reduction.

6.4. HYBRID OPTIMIZATION PIPELINE

The hybrid optimization pipeline, designed to balance model compression with real-time performance, is detailed



in the flowchart below. The process initiates with the standard steps of pruning and weight clustering applied to the trained model. From this clustered state, the pipeline attempts a direct path to full quantization by applying ONNX optimizations to generate both a simplified FP32 model and a fully quantized static INT8 model. If the static INT8 model retains an accuracy above a 90% threshold, the optimization is considered successful and terminated. Otherwise we proceed with the partial quantization of the original clustered model using TFLite Converter and pass this partially quantized model through ONNX Optimization Pipeline. By doing so we were able to engineer highly

compressed models (extremely reduced size) with high improvement on throughput and latency. This hybrid, conditional strategy successfully engineers highly compressed models with substantial improvements in both throughput and latency.

The complete implementation of this conditional optimization logic is open-sourced and accessible in our public repository [\[Model Optimization\]](#).

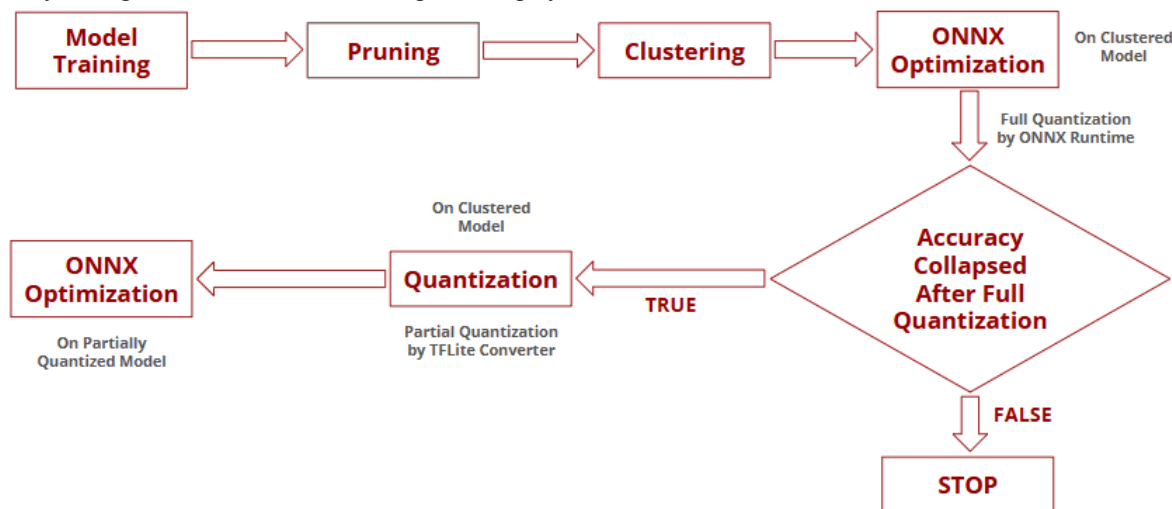


Figure 5: Flowchart of the Hybrid Model Optimization Pipeline

6.5. TVM MIGRATION TO SHAKTI

A complete TVM-based embedded AI deployment pipeline was developed for the Shakti RISC-V platform. ONNX models such as MobileNet and ResNet are imported into TVM Relay, optimized using graph-level and mixed-precision transformations, and compiled using TVM's AOT flow to generate standalone C code. A unified Makefile-based build system was developed to integrate BSP drivers, TVM CRT runtime, generated model code, linker scripts, and application sources into a single dependency-driven flow. The system supports both x86 host execution for functional validation and RISC-V cross-compilation for FPGA deployment on Shakti targets such as Ganga and Yamuna, while also enabling future hardware accelerator integration within the TVM compilation framework.



Figure 6: The TVM Migration Pipeline

6.6. CONV ACCELERATOR

A custom FPGA-based convolution accelerator was developed and integrated into a TVM-generated inference pipeline targeting the Shakti RISC-V platform. The accelerator is designed to offload computationally intensive convolution operations commonly used in lightweight CNN models. Using a pipelined SIMD MAC architecture with AXI-based memory transfers, the hardware autonomously performs input fetch, convolution computation, accumulation, and output writeback, significantly reducing software loop overhead on the CPU. By accelerating convolution-heavy layers directly in hardware while allowing unsupported operations to continue executing on the processor, the design enables a heterogeneous inference pipeline capable of improving throughput and reducing latency for embedded edge-AI workloads.

6.7 FPGA DEPLOYMENT AND EXECUTION FLOW

After generating the FPGA bitstream and integrating the TVM-generated runtime, the complete deployment flow is executed on the Shakti RISC-V FPGA platform. The deployment process consists of FPGA programming, application preparation, TVM model integration, host-side



validation, and final accelerator-assisted execution on the FPGA platform.

6.8. FPGA PROGRAMMING

The FPGA hardware is synthesized and built using the hardware build flow for the Nexys Video platform. The generated FPGA bitstream is programmed onto the FPGA board using Vivado Hardware Manager. This configures the Shakti RISC-V SoC together with the integrated convolution accelerator on the FPGA fabric.

6.9. IMAGE PREPARATION AND C ARRAY GENERATION

Inference images are converted into embedded C arrays before deployment. This allows static image data to be directly linked into the embedded application binary.

The image conversion utility is located in the TVM utility scripts directory. Before execution, the required image path is updated inside the Python script. The conversion script generates an **image_data.h** header file containing the image represented as a C array.

The generated header file is copied into the application source directory, where it is included during application compilation and inference execution.

6.10. TVM MODEL INTEGRATION

The trained ONNX model is imported into TVM Relay and compiled using TVM's Ahead-of-Time (AOT) compilation flow. The generated C kernels and runtime wrappers are produced using the TVM build scripts.

Before running the TVM scripts, the TVM environment variables are configured to point to the local TVM installation. The generated C output from the TVM Python scripts is copied into the generated model directory used by the build system.

The generated model directories include:

- model_acc
- model_norm
- model_int
- mobilenet_model
- resnet_model

6.11. APPLICATION BUILD FLOW

A unified Makefile-based build system is used for:

- Host execution
- Shakti CPU execution
- Accelerator-assisted execution

The Makefile supports configurable model selection using the **MODEL** parameter.

This unified build flow integrates:

- BSP drivers
- TVM CRT runtime
- Generated model kernels
- Application sources
- Linker scripts
- Accelerator communication logic

Into a single dependency-driven compilation framework.

6.12. HOST-SIDE FUNCTIONAL VALIDATION

Before FPGA deployment, the generated model and runtime are validated on the host system. This step is used to:

- Validate generated TVM kernels
- Verify runtime correctness
- Check inference outputs
- Debug model integration before hardware deployment

6.13. FPGA RUNTIME EXECUTION

The FPGA execution flow uses OpenOCD and GDB to load and execute the generated Shakti executable on the FPGA platform.

Both accelerator-enabled execution and CPU-only execution are supported. The executable is transferred into FPGA memory, validated, and executed through the debugging interface.

6.14. ACCELERATOR RUNTIME EXECUTION

During execution, convolution layers are selectively offloaded to the FPGA accelerator.

Example runtime logs include execution parameters such as:

- Kernel size (K)
- Accelerator execution mode (MODE)
- Input tensor size
- Accelerator completion status

KERNEL SIZE (K)

VALUE	MEANING
K=1	1×1 pointwise convolution
K=3	3×3 standard convolution



ACCELERATOR MODE (MODE)

VALUE	MEANING
MODE =0	Standard convolution mode
MODE=2	Pointwise/optimized convolution mode

The accelerator autonomously performs:

- Input fetch
- MAC computation
- Accumulation
- Output writeback

While unsupported operations continue executing on the Shakti processor.

6.15. OUTPUT VALIDATION AND ACCELERATOR STATUS

The current accelerator implementation is still under development, and observable numerical inconsistencies exist between CPU execution and accelerator execution.

At present, the exact source of these discrepancies has not been conclusively identified. Possible causes may include:

- Fixed-point arithmetic behavior
- Accumulation ordering differences
- Quantization effects
- Memory alignment issues
- Convolution scheduling mismatches
- Runtime integration behavior

Further debugging and layer-wise validation are required to accurately determine the root cause of the discrepancy.

Despite these inconsistencies, the accelerator flow currently succeeds in:

- Invoking hardware execution successfully
- Transferring tensors correctly
- Completing end-to-end inference execution
- Generating final classification outputs

The current accelerator should therefore be considered a functional prototype intended primarily for integration, experimentation, and validation purposes rather than a fully verified inference accelerator implementation.

A practical demonstration of the system's execution, with the accelerator output is shown in the upcoming [\[Final Demo Video\]](#). It is sped up 3x times than real life.

7. RESULTS

This section presents the performance evaluation of five models optimized using the hybrid optimization pipeline. Each model was benchmarked against five key metrics: throughput (images per second), latency (milliseconds per inference), classification accuracy, raw model size (MB), and compressed (zipped) model size. The compressed size is included as a specific metric to quantify the impact of weight pruning and clustering on the size of the models we have trained so far. The comprehensive results are detailed in the performance tables shown in this section..

7.1. FULLY QUANTIZED MODELS

Table 1: Performance evaluation of the RTDD architecture [2] across the optimization pipeline stages

RTDD (IEEE)	Throughput	Accuracy	Model Size (Raw)	Model Size (ZIP)	Latency
Original	25.2 imgs/sec	99.90%	337.18 MB	312.66 MB	39.69 ms
Pruning	135.3 imgs/sec	99.81%	112.42 MB	39.20 MB	7.39 ms
Clustering	135.1 imgs/sec	99.76%	112.41 MB	7.94 MB	7.40 ms
Simplified FP32 ONNX	6.08 imgs/sec	99.79%	112.37 MB	39.19 MB	164.57 ms
Fully Quantized ONNX	7.06 imgs/sec	99.79%	28.13 MB	10.76 MB	141.63 ms

Table 2: Performance evaluation of the ResNet-18 architecture across the optimization pipeline stages

ResNet - 18	Throughput	Accuracy	Model Size (Raw)	Model Size (ZIP)	Latency
Original	25.9 imgs/sec	99.86%	128.40 MB	118.27 MB	38.67 ms
Pruning	25.7 imgs/sec	99.86%	42.89 MB	39.61 MB	36.37 ms
Clustering	45.8 imgs/sec	99.90%	42.87 MB	9.39 MB	21.84 ms
Simplified FP32 ONNX	56.44 imgs/sec	99.58%	42.64 MB	39.53 MB	17.72 ms
Fully Quantized ONNX	76.44 imgs/sec	99.79%	10.72 MB	10.15 MB	13.08 ms

The RTDD architecture, adapted from Reddy et al. [2], was successfully compressed from an initial 337.18 MB to a 28.13 MB Fully Quantized ONNX model, retaining an excellent accuracy of 99.79%. This static ONNX model delivered 7.06 images/sec with a 141.63 ms latency, proving vastly superior to the TFLite-based partial



quantization fallback. Specifically, the TFLite partial quantization proved completely unviable for this architecture, suffering a catastrophic drop to 51.88% accuracy and severe latency overhead (1730.81 ms), validating the ONNX path as the optimal strategy.

ResNet-18 demonstrated exceptional compatibility with the fully quantized pipeline, achieving a raw size reduction from 128.40 MB to 10.72 MB while maintaining a near-lossless accuracy of 99.79%. Crucially, temporal performance improved drastically across the optimization stages, with the final ONNX model nearly tripling initial throughput to 76.44 images per second and reducing latency to just 13.08 ms.

7.2. PARTIALLY QUANTIZED MODELS

The MobileNetV3 Small architecture was ultimately selected as the optimal candidate for FPGA deployment due to its superior balance of spatial efficiency and temporal performance. Because full static ONNX quantization induced a catastrophic accuracy drop to 47.16%, the conditional partial quantization fallback was engaged. The resulting ONNX-optimized partial model achieved a highly compressed raw size of just 3.18 MB while successfully recovering accuracy to 96.88%. This final deployed configuration nearly quadrupled the original throughput to 80.38 images per second and reduced latency to 12.44 ms, easily meeting the strict requirements for real-time hardware execution.

Table 3: Performance evaluation of the MobileNetV3 Small architecture across the optimization pipeline stages

MobileNetV3 Small	Throughput	Accuracy	Model Size (Raw)	Model Size (ZIP)	Latency
Original	24.1 Imgs/sec	99.14%	31.20 MB	28.24 MB	41.54 ms
Pruning	16.7 Imgs/sec	99.14%	12.50 MB	7.09 MB	59.73 ms
Clustering	15.6 Imgs/sec	98.90%	12.38 MB	1.90 MB	64.17 ms
Partial Quantization	9.73 Imgs/sec	93.83%	3.22 MB	1.53 MB	102.73 ms
Simplified FP32 ONNX	80.38 Imgs/sec	96.88%	3.18 MB	1.48 MB	12.44 ms
Fully Quantized ONNX	202.52 Imgs/sec	47.16%	3.21 MB	1.49 MB	4.94 ms

Similar to its smaller counterpart, the MobileNetV3 Large architecture suffered a severe accuracy collapse (52.92%) under full static quantization, necessitating the partial quantization fallback route. By routing the compressed weights through the simplified ONNX optimization pipeline, the model achieved a perfect 100.00% accuracy

while successfully reducing its raw spatial footprint from 47.43 MB to 6.05 MB. Furthermore, this hybrid approach bypassed the extreme latency penalties associated with standard TFLite partial quantization, ultimately delivering a stable throughput of 45.43 images per second at a 22.02 ms latency.

Table 4: Performance evaluation of the MobileNetV3 Large architecture across the optimization pipeline stages

MobileNetV3 Large	Throughput	Accuracy	Model Size (Raw)	Model Size (ZIP)	Latency
Original	24.7 Imgs/sec	98.76%	47.43 MB	41.17 MB	40.56 ms
Pruning	14.9 Imgs/sec	98.76%	23.96 MB	16.95 MB	67.30 ms
Clustering	14.8 Imgs/sec	99.90%	23.81 MB	4.24 MB	67.36 ms
Partial Quantization	3.17 Imgs/sec	76.37%	6.22 MB	3.63 MB	315.50 ms
Simplified FP32 ONNX	45.43 Imgs/sec	100.00%	6.05 MB	3.50 MB	22.02 ms
Fully Quantized ONNX	119.61 Imgs/sec	52.92%	6.07 MB	3.26 MB	8.36 ms

Table 5: Performance evaluation of the MobileNetV2 architecture across the optimization pipeline stages

MobileNetV2	Throughput	Accuracy	Model Size (Raw)	Model Size (ZIP)	Latency
Original	22.8 Imgs/sec	99.90%	760.64 MB	704.42 MB	43.83 ms
Pruning	21.60 Imgs/sec	99.90%	256.12 MB	73.58 MB	46.25 ms
Clustering	22.60 Imgs/sec	98.04%	256.07 MB	16.15 MB	44.44 ms
Partial Quantization	1.3 Imgs/sec	99.69%	64.28 MB	N/A	741.40 ms
Simplified FP32 ONNX	15.60 Imgs/sec	95.31%	64.07 MB	11.84 MB	64.12 ms
Fully Quantized ONNX	10.28 Imgs/sec	52.19%	64.05 MB	11.80 MB	97.26 ms

MobileNetV2 exhibited similar quantization instability, suffering a catastrophic accuracy drop to 52.19% under the fully quantized ONNX path, thereby triggering the partial quantization fallback. The hybrid optimization pipeline successfully condensed the massive 760.64 MB original raw model down to 64.07 MB while preserving an acceptable 95.31% classification accuracy. However, unlike the MobileNetV3 variants, this extreme spatial compression came at a temporal cost; the final deployed model saw



throughput degrade to 15.60 images per second and latency increase to 64.12 ms relative to the unoptimized baseline.

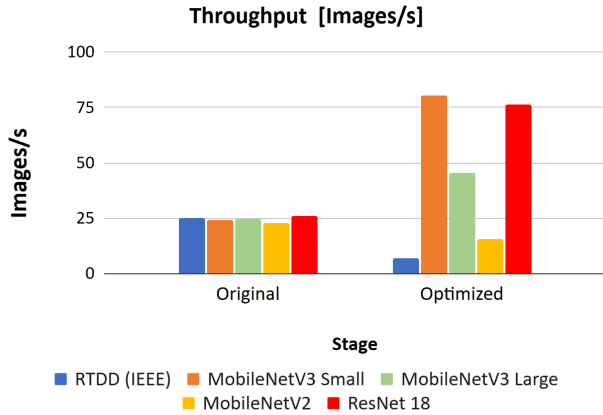


Figure 7: Throughput (images/s) of the five deep models before and after optimization

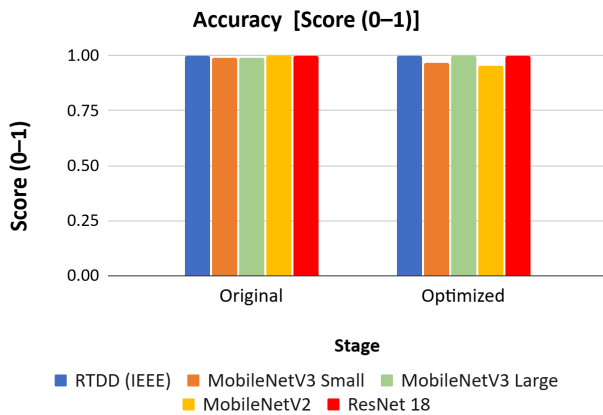


Figure 8: Accuracy of five models before and after optimization

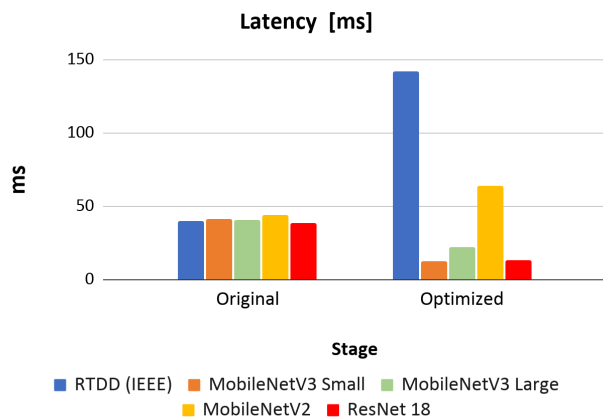


Figure 9: Latency (ms) of the five models before and after optimization

7.3. OVERALL PERFORMANCE EVALUATION

The evaluation of five distinct architectures through the hybrid optimization pipeline confirmed the critical

trade-offs between model compression, inference latency, and quantization stability. While ResNet-18 proved to be the most robust candidate for full static quantization, MobileNet-based architectures demonstrated significant sensitivity, necessitating the conditional partial quantization fallback. The MobileNetV3-Small model emerged as the most suitable candidate for deployment on our FPGA device, achieving a near-optimal balance with an 8.7x raw size reduction and a 3.3x latency improvement while maintaining a viable 96.88% accuracy. This comprehensive analysis validates the hybrid optimization pipeline as an effective methodology for engineering hardware-compatible models under complex, real-world constraints.

7.4. MODEL SELECTION FOR FPGA DEPLOYMENT

Although MobileNetV3 Small achieved the best overall balance in terms of throughput, latency, and compressed model size, ResNet-18 was selected as the primary candidate for FPGA accelerator integration on the Shakti platform. The decision was motivated primarily by hardware design considerations rather than software-only inference performance.

The MobileNetV3 family relies heavily on floating-point operations, depthwise separable convolutions, activation scaling, and more complex arithmetic behavior, which significantly increases the hardware complexity of accelerator implementation. Efficient deployment of MobileNetV3 on FPGA hardware generally requires floating-point multiply-accumulate (FP MAC) units or highly optimized mixed-precision datapaths.

Such designs increase:

- Hardware area utilization
- Routing complexity
- DSP utilization
- Power consumption
- Verification complexity

In contrast, ResNet-18 provides a comparatively simpler convolution structure that maps more efficiently onto lightweight fixed-point FPGA accelerators. The architecture primarily relies on standard convolution layers and regular tensor layouts, making it substantially easier to integrate using integer-based MAC datapaths without requiring floating-point hardware support.

As a result, ResNet-18 enables:

- Simpler accelerator control logic
- Lower FPGA resource utilization
- Reduced hardware verification complexity
- Easier memory scheduling
- Improved compatibility with fixed-point arithmetic pipelines

This trade-off between hardware simplicity and inference performance made ResNet-18 a more practical choice for



early-stage accelerator integration and FPGA validation on the Shakti RISC-V platform.

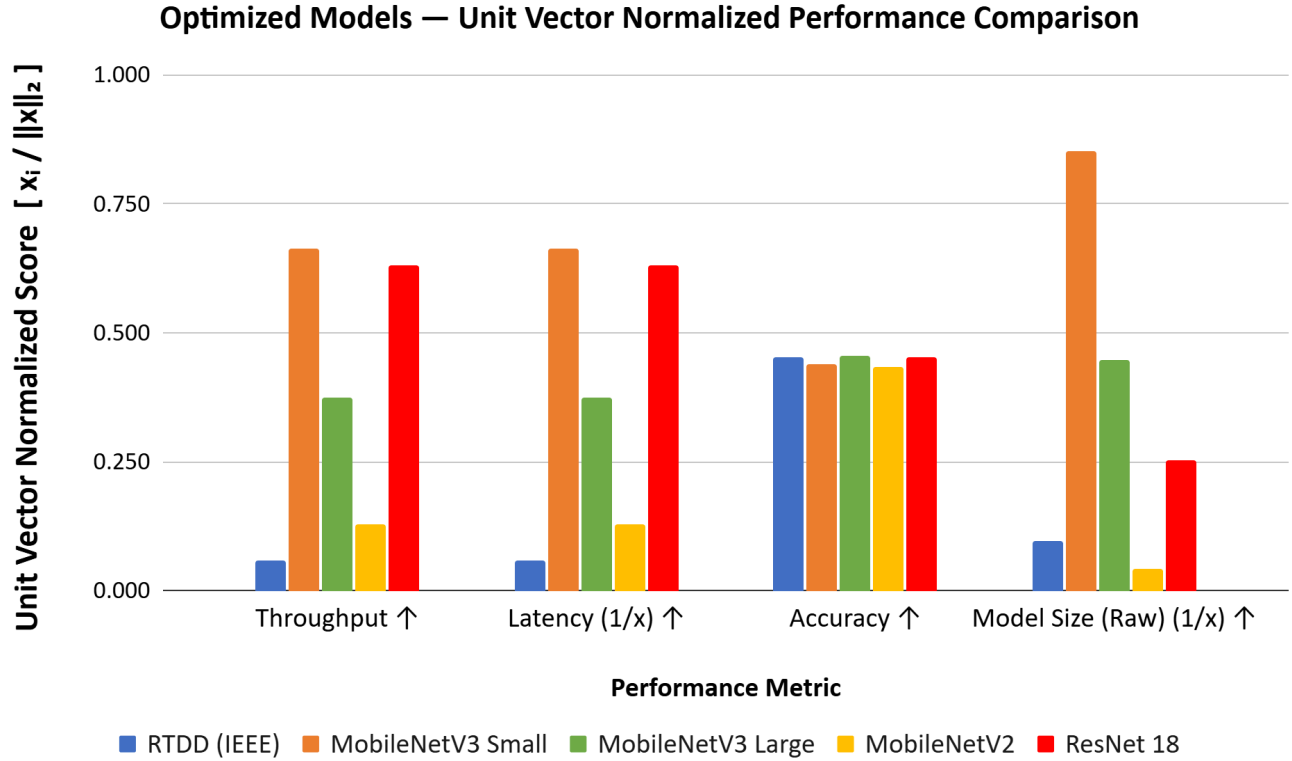


Figure 10: Unit vector normalized comparison of the five optimized models. Latency and Model Size are reciprocated before normalization so that a taller bar consistently indicates better performance

8. DISCUSSION

The results demonstrated the successful development of a hybrid optimization pipeline combining TFMOT and ONNX optimization techniques. This unified approach overcame the limitations of using either method independently and enabled the generation of balanced, highly compressed models suitable for edge deployment.

MobileNetV3 Small emerged as the best software-side model, achieving 96.88% accuracy within a 3.18 MB raw footprint (1.48 MB compressed), while delivering real-time performance at 12.44 ms latency and 80.38 FPS throughput.

However, ResNet-18 was selected as the primary candidate for FPGA accelerator integration due to hardware design considerations. Unlike MobileNetV3, which relies on more complex floating-point-intensive operations, ResNet-18 maps more efficiently onto lightweight fixed-point FPGA accelerators without requiring floating-point MAC support.

This results in:

- Lower hardware area utilization
- Simpler accelerator design
- Easier fixed-point implementation
- Reduced verification complexity

As a result, ResNet-18 provided a more practical balance between inference capability and FPGA implementation feasibility for accelerator integration on the Shakti RISC-V platform.

9. CONCLUSION

This work developed an FPGA-based real-time driver drowsiness detection system using optimized lightweight deep learning models on a Shakti RISC-V platform. A hybrid TFMOT–ONNX optimization pipeline achieved an effective balance between accuracy, latency, and model size. The system also integrated TVM-based deployment, and FPGA convolution acceleration, for autonomous intervention during driver drowsiness. The results demonstrate the feasibility of efficient edge-AI deployment for real-time automotive safety applications.



10. CODE AVAILABILITY

To support reproducibility and open science, the complete codebase for this hardware-software co-design project has been open-sourced across two repositories:

- Model Optimization Pipeline (Kaggle): The source code for the hybrid optimization pipeline is available at:
<https://www.kaggle.com/code/rogaldorn7/drowsiness-detection-model-optimization>
- Hardware Deployment (GitHub): The C/C++ implementation, RISC-V cross-compilation scripts, and FPGA execution environment are available at:
<https://github.com/Harikheshav/Drowsiness-Detection>

REFERENCES

- [1] S. K. Mousavikia, E. Gholizadehazari, M. Mousazadeh and S. B. Örs Yalçın, "Instruction Set Extension of a RISC-V Based SoC for Driver Drowsiness Detection," *IEEE Access*, vol. 10, pp. 58151–58162, May 2022, doi: 10.1109/ACCESS.2022.3177743.
- [2] B. Reddy, Y.-H. Kim, S. Yun, C. Seo and J. Jang, "Real-Time Driver Drowsiness Detection for Embedded System Using Model Compression of Deep Neural Networks," *2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Honolulu, HI, USA, 2017, pp. 438–445, doi: 10.1109/CVPRW.2017.59.
- [3] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez and V. Koltun, "CARLA: An Open Urban Driving Simulator," *Proceedings of the 1st Conference on Robot Learning (CoRL)*, 2017, pp. 1–16.
- [4] I. Nasri, "Driver Drowsiness Dataset (DDD)," Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/ismailnasri20/driver-drowsiness-dataset-ddd/data>. Accessed: Mar. 2026.
- [5] E. M. Sayed, "Driver Drowsiness Detection (CNN/MobileNetV2)," Kaggle, 2024. [Online]. Available: <https://www.kaggle.com/code/esraameslamsayed/driver-drowsiness-detection-cnn-mobilenetv2>.